Empleados

Pepe Calderón, Fernando Sola, Daniel Ayala, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz

Contenido

1. Requisitos . . . 2
2. Requisitos de información (RI) . . . 2
   2.1. RI-01: Departamentos . . . 2
   2.2. RI-02: Empleados . . . 2
3. Reglas de negocio (RN) . . . 2
   3.1. RN-01: Departamentos . . . 2
   3.2. RN-02: Empleados . . . 2
4. Modelo Conceptual . . . 2
5. Diagrama de clases . . . 2
6. Diagrama de objetos . . . 3
7. Modelo Relacional . . . 3
8. Intensión . . . 4
9. Extensión . . . 4
10. Álgebra relacional . . . 4
11. Modelo Tecnológico (MariaDB) . . . 5
12. Script SQL para crear la base de datos . . . 5
13. Script SQL para la carga inicial de datos . . . 6
14. Consultas . . . 7
15. SQL Avanzado . . . 10
   15.1. Procedimientos almacenados . . . 10
   15.2. Funciones . . . 12
   15.3. Cursores . . . 13
   15.4. Triggers (disparadores) . . . 14

Versión PDF disponible

Empleados

1. Requisitos

Catálogo de Requisitos Se pretende realizar un pequeño sistema de información para gestionar los empleados de los departamentos de una empresa. Cada empleado pertenece a un departamento y puede tener un jefe. Cada departamento pertenece a una localidad. Cada empleado tiene un salario y una comisión, además se almacena la fecha de inicio y de finalización de su contrato.

2. Requisitos de información (RI)

2.1. RI-01: Departamentos

- **Como:** Profesor de la asignatura
- **Quiero:** Poder almacenar la siguiente información de los departamentos: nombre del departamento(obligatorio) y localidad(opcional)
- **Para:** Que el alumno practique con un sistema de información sencillo

2.2. RI-02: Empleados

- **Como:** Profesor de la asignatura
- **Quiero:** Poder almacenar la siguiente información de los empleados: nombre (obligatorio y único), salario, comisión, fecha de inicio y finalización del contrato (opcionales), jefe (opcional) y departamento al que pertenece (obligatorio)
- **Para:** Que el alumno practique con un sistema de información sencillo

3. Reglas de negocio (RN)

3.1. RN-01: Departamentos

- **Como:** Profesor de la asignatura
- **Quiero:** Un departamento no puede tener más de cinco empleados y la combinación nombre y localidad no puede repetirse
- **Para:** Que el alumno practique con restricciones simples

3.2. RN-02: Empleados

- **Como:** Profesor de la asignatura
- **Quiero:** Un empleado no puede ser jefe de si mismo, la comisión es un porcentaje del salario y no puede ser negativa, ni modificarse en más de un 20% de golpe
- **Para:** Que el alumno practique con restricciones simples

4. Modelo Conceptual

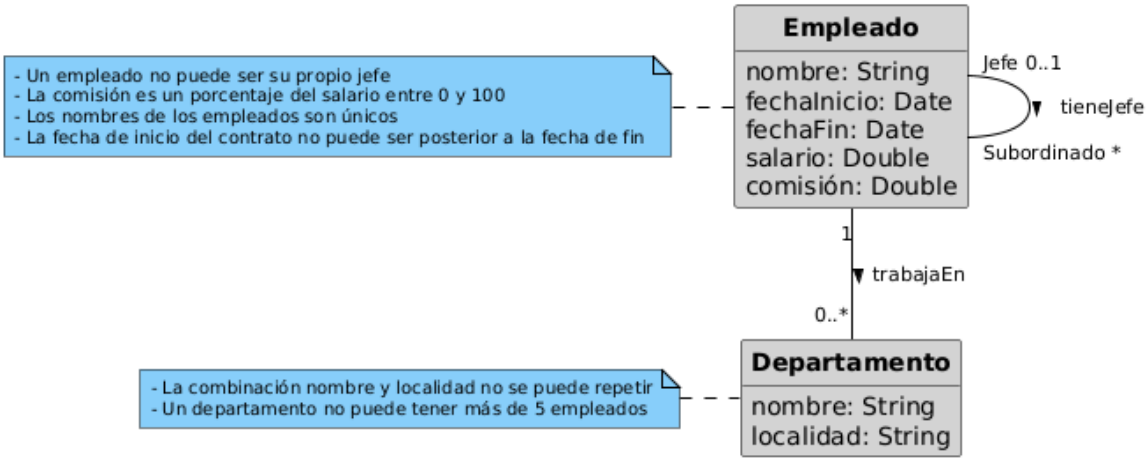
Modelo conceptual

5. Diagrama de clases

El modelo representa una estructura organizacional con empleados distribuidos en departamentos.

- **Entidades principales**: Empleado, Departamento.

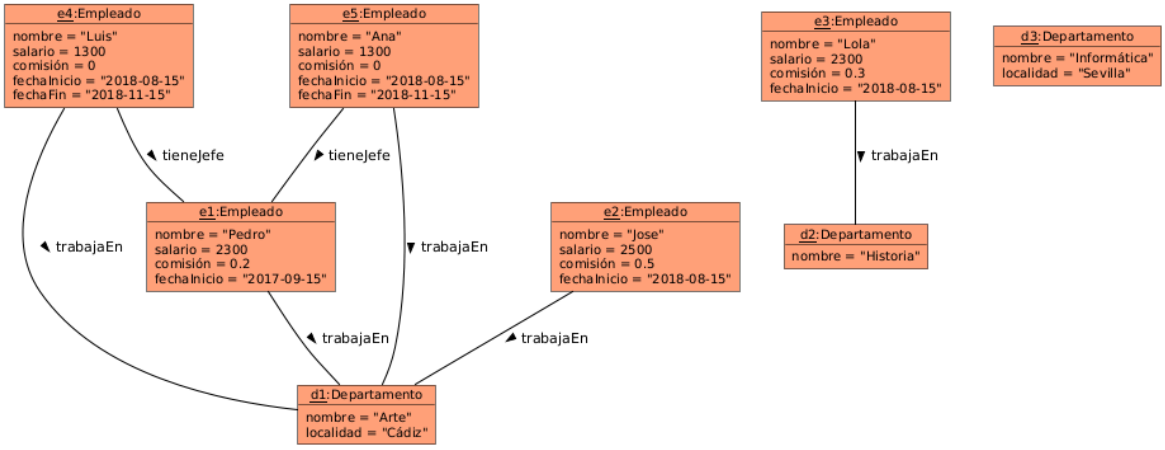
- **Asociaciones**: Un Empleado pertenece a 1 Departamento; un Departamento tiene 1..* Empleados.
- **Roles**: Los empleados puedes ser Jefes o Subordinados, de forma que un Subordinado tiene 0..1 Jefes y un Jefe tiene 0..* Subordinados
- **Restricciones**: un empleado no puede ser su propio jefe, la comisión es un porcentaje, los nombres de los empleados son únicos, las fechas de inicio y fin de contrato deben ser coherentes, la tupla (nombre, localidad) de un departamento no puede repetirse, y los departamentos no pueden tener más de 5 empleados.



6. Diagrama de objetos

Las instancias muestran la diversidad de empleados y la organización departamental de la universidad.

- **Departamentos**: Diferentes departamentos múltiples empleados asignados (d1, d2, d3).
- **Empleados**: Cinco empleados, Juan es Jefe de Luis y Ana, José y Lola no tiene jeje, y el departamento d3 no tiene ningún empleado.



7. Modelo Relacional

Modelo relacional

8. Intensión

```
Departamentos(departamentoId, nombre, localidad)
  PK(departamentoId)
  AK(nombre, localidad)
Empleados(empleadoId, departamentoId, jefeId, nombre, salario, fechaInicio,
fechaFin, comision)
  PK(empleadoId)
  FK(departamentoId) / Departamentos
  FK(jefeId) / Empleados
  AK(nombre)
```

9. Extensión

```
Departamentos = {
  (d1, 'Arte', 'Cádiz'),
  (d2, 'Historia', null),
  (d3, 'Informática', 'Sevilla')
}
Empleados = {
  (e1, d1, NULL, 'Pedro', 2300.00, '2017-09-15', NULL, 0.2),
  (e2, d1, NULL, 'Jose', 2500.00, '2018-08-15', NULL, 0.5),
  (e3, d2, NULL, 'Lola', 2300.00, '2018-08-15', NULL, 0.3),
  (e4, d1, e1, 'Luis', 1300.00, '2018-08-15', '2018-11-15', 0),
  (e5, d1, e1, 'Ana', 1300.00, '2018-08-15', '2018-11-15', 0)
}
```

10. Álgebra relacional

- Renombrado de relaciones para acortar las expresiones en álgebra relacional:

$$\underset{D(dId,n,l)}{\rho}(Departamentos)$$

$$\underset{E(eId,dId,jId,n,s,FI,Ff,c)}{\rho}(Empleados)$$

- Empleados con sueldo < 2000:

$$\underset{s<2000}{\sigma}(E)$$

- Fechas de alta y baja:

$$\underset{fI,fF}{\Pi}(E)$$

- Sueldo entre 2000 y 3000:

$$\underset{2000<s<3000}{\sigma}(E)$$

- Producto cartesiano:

$$E \times D$$

- Join Empleados–Departamentos:

$$E \bowtie D$$

- Departamentos con empleados:

$$\underset{dId}{\Pi}(E)$$

- Departamentos sin empleados:

$$\underset{dId}{\Pi}(D) \setminus \underset{dId}{\Pi}(E)$$

- Estadísticas globales de salario:

$$\underset{\gamma}{\overset{\text{COUNT}(*),\ \text{MIN}(s),\ \text{MAX}(s),\ \text{AVG}(s),\ \text{SUM}(s)}{}}(E)$$

- Estadísticas de salario por departamento:

$$\underset{\gamma}{\overset{\text{COUNT}(*),\ \text{MIN}(s),\ \text{MAX}(s),\ \text{AVG}(s),\ \text{SUM}(s)}{}} dId(E)$$

- Estadísticas de salarios por departamento con al menos dos empleados:

$$Dep2 \leftarrow \underset{dId}{\Pi}\left(\underset{\text{COUNT}(*)\geq 2}{\sigma}\left(\underset{\gamma}{\overset{\text{COUNT}(*)}{}} dId(E)\right)\right)$$

$$\underset{\gamma}{\overset{\text{COUNT}(*),\ \text{MIN}(s),\ \text{MAX}(s),\ \text{AVG}(s),\ \text{SUM}(s)}{}} dId(E \bowtie Dep2)$$

11. Modelo Tecnológico (MariaDB)

Modelo tecnológico (MariaDB)

12. Script SQL para crear la base de datos

Empleados/createDB.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Script para crear la BD
--

DROP DATABASE if EXISTS EmployeesDB;
CREATE DATABASE EmployeesDB;
USE EmployeesDB;

DELIMITER //
CREATE OR REPLACE PROCEDURE pCreateDB()
BEGIN
  CREATE OR REPLACE TABLE Departments (
    departmentId INT NOT NULL AUTO_INCREMENT,
    nameDep VARCHAR(32),
    city VARCHAR(64),
    UNIQUE (nameDep, city),
    PRIMARY KEY(departmentId)
  );

  CREATE OR REPLACE TABLE Employees (
    employeeId INT NOT NULL AUTO_INCREMENT ,
    departmentId INT,
    bossId INT,
    nameEmp VARCHAR(64) NOT NULL,
    startDate DATE,
```

```
    endDAte DATE,
    salary DOUBLE DEFAULT 2000,
    fee DOUBLE,
    PRIMARY KEY(employeeId),
    FOREIGN KEY (departmentId) REFERENCES Departments (departmentId)
      ON DELETE RESTRICT,
    FOREIGN KEY (bossId) REFERENCES Employees (employeeId),
    UNIQUE(nameEmp),
    CONSTRAINT validFee CHECK (fee>=0 AND fee<=1),
    CONSTRAINT valifDates CHECK (startDate < endDate),
    CONSTRAINT validSalary CHECK (salary > 0)
  );
END;
//
DELIMITER ;

CALL pCreateDB();
```

13. Script SQL para la carga inicial de datos

Empleados/populateDB.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Script para la carga inicial de datos
--

DELIMITER //
CREATE OR REPLACE PROCEDURE pPopulateDB()
BEGIN
  -- desactivar las restricciones de FK para poder borrar, la tabla de empleados
tiene una FK a si misma.
  SET FOREIGN_KEY_CHECKS = 0;
  DELETE FROM Employees;
  DELETE FROM Departments;
  SET FOREIGN_KEY_CHECKS = 1;

  INSERT INTO Departments (departmentId, nameDep, city) VALUES
    (1, 'Arte', 'Cádiz'),
    (2, 'Historia', NULL),
    (3, 'Informática', 'Sevilla')
  ;

  INSERT INTO Employees (employeeId, departmentId, bossId, nameEmp, salary,
startDate, endDate, fee) VALUES
    (1, 1, NULL, 'Pedro', 2300.00, '2017-09-15', NULL, 0.2),
    (2, 1, NULL, 'Jose', 2500.00, '2018-08-15', NULL, 0.5),
    (3, 2, NULL, 'Lola', 2300.00, '2018-08-15', NULL, 0.3),
    (4, 1, 1, 'Luis', 1300.00, '2018-08-15', '2018-11-15', 0),
    (5, 1, 1, 'Ana', 1300.00, '2018-08-15', '2018-11-15', 0)
  ;
END;
//
DELIMITER ;

CALL pPopulateDB();
```

14. Consultas

Empleados/queries.sql

```sql
/* Consultas, para probar las consultas una a una debe ejecutar
Shift+Control+F9  con el cursor en la línea que contiene la consulta */

/* Employees con un sueldo inferior a 2000 euros */
SELECT *
FROM Employees
WHERE salary < 2000;

/* Employees con un sueldo inferior a 2000 euros */
SELECT nameEmp, salary
FROM Employees
WHERE salary < 2000;

/* Fechas de alta y baja de los empeados como una lista */
SELECT ALL startDate, endDate
FROM Employees;

/* Fechas de alta y baja de los empeados como un conjunto */
SELECT DISTINCT startDate, endDate
FROM Employees;

/* Conjunto de Employees con un salary en [2000,3000] */
/* Opción 1 */
SELECT DISTINCT nameEmp, salary
FROM Employees
WHERE salary >=2000 AND salary <=3000;
/* Opción 2 */
SELECT DISTINCT nameEmp, salary
FROM Employees
WHERE salary BETWEEN 2000 AND 3000;

/* Conjunto de Employees con salary de 1000, 2500 o 3000 euros */
SELECT DISTINCT nameEmp, salary
FROM Employees
WHERE salary IN (1000,2500,3000);

/* Lista de Employees con una 'o' en la segunda posición de su nameEmp
o que son jefes */
SELECT *
FROM Employees
WHERE nameEmp LIKE '_o%' OR bossId IS NULL;

/* Lista de Employees ordenada por departmentId y luego por nameEmp */
SELECT *
FROM Employees
ORDER BY departmentId, nameEmp ASC;

/* Producto cartesiano de Employees y Departments */
SELECT *
FROM Employees, Departments;

/* Employees y Departments en los que trabajan. Natural join */
/* Opción 1: */
```

```sql
SELECT nameEmp, salary, startDate, nameDep
FROM Employees E, Departments D
WHERE E.departmentId=D.departmentId;
/* Opción 2: */
SELECT nameEmp, salary, startDate, nameDep
FROM Employees NATURAL JOIN Departments;

/* Join parciales */
/* Para hacer una prueba borramos el departamento de Ana */
UPDATE Employees SET departmentId=NULL WHERE employeeId=5;

/* En el left join se devuelve a Ana, aunque no tenga departamento */
SELECT nameEmp, salary, startDate, nameDep
FROM Employees E
  LEFT JOIN Departments D
  ON E.departmentId=D.departmentId;

/* En el right join se devuelve el Departamento de Arte, aunque no tenga Employees
*/
SELECT nameEmp, salary, startDate, nameDep
FROM Employees E
  RIGHT JOIN Departments D
  ON E.departmentId=D.departmentId;

/* Ejemplo de unión de left y right join, devuelve el full join */
SELECT *
FROM Employees E
  LEFT JOIN Departments D
  ON E.departmentId=D.departmentId
UNION
SELECT *
FROM Employees E
  RIGHT JOIN Departments D
  ON E.departmentId=D.departmentId;

/* Departments sin Employees */
SELECT *
FROM Departments D
WHERE NOT EXISTS (
  SELECT * FROM Employees E
  WHERE D.departmentId=E.departmentId
);

/* Departments con Employees */
SELECT *
FROM Departments D
WHERE EXISTS (
  SELECT * FROM Employees E
  WHERE D.departmentId=E.departmentId
);

/* Estadísticas salarys de los Employees */
SELECT COUNT(*), MIN(salary), MAX(salary), AVG(salary), SUM(salary)
FROM Employees;

/* Estadísticas salarys por departamento */
```

```sql
SELECT departmentId,
  COUNT(*),
  AVG(salary) angSalary,
  AVG(salary * (1+fee)) salaryConfee,
  SUM(salary) gastosalarys
FROM Employees
GROUP BY departmentId;

/* Estadísticas salarys por departamento con al menos dos empleado*/
SELECT departmentId,
  COUNT(*),
  AVG(salary) salaryMedio,
  AVG(salary * (1+fee)) salaryConfee,
  SUM(salary) gastosalarys
FROM Employees
GROUP BY departmentId HAVING COUNT(*)>1;
/* Opción 2: Usando la vista EmployeesDepartments */
CREATE OR REPLACE VIEW vEmployeesDepartments AS
SELECT *
FROM Employees NATURAL JOIN Departments;

SELECT nameDep,
  COUNT(*),
  AVG(salary) salaryMedio,
  AVG(salary * (1+fee)) salaryConfee,
  SUM(salary) gastosalarys
FROM vEmployeesDepartments
GROUP BY departmentId HAVING COUNT(*)>1;

/* Employees con salary mayor que la media de su departamento*/
SELECT * FROM Employees
WHERE salary >
ALL (SELECT AVG(salary)
       FROM Employees
       GROUP BY departmentId);

/* Departamento con más Employees */
/* Opción 1 */
SELECT departmentId FROM Employees
GROUP BY departmentId HAVING COUNT(*)>= ALL
   ( SELECT COUNT(*)
     FROM Employees
     GROUP BY departmentId
    );

/* Opción 2 */
SELECT departmentId FROM Employees
GROUP BY departmentId HAVING COUNT(*) =
   ( SELECT MAX(total) FROM
      ( SELECT COUNT(*) AS total
        FROM Employees
        GROUP BY departmentId
       ) NumEmployees
   );
```

```
/* Vista con las estadísticas de los Employees por Departamento */
CREATE OR REPLACE VIEW vStatEmployees AS
SELECT departmentId,
  COUNT(*) AS numEmployees,
  AVG(salary) salaryMedio,
  AVG(salary * (1+fee)) salaryWithfee,
  SUM(salary) totalSalary
FROM Employees
GROUP BY departmentId;

/* Número de Employees que tiene el departamento con más Employees */
SELECT MAX(numEmployees)
FROM vStatEmployees;
```

15. SQL Avanzado

15.1. Procedimientos almacenados

Realice procedimientos para insertar en las tablas Departments y Employees:

Empleados/pInsertDepartment.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Procedimiento para insertar un nuevo Departamento
--

DELIMITER //
CREATE OR REPLACE PROCEDURE
  pInsertDepartment(
    p_nameDep VARCHAR(32),
    p_city VARCHAR(64))
BEGIN
  INSERT INTO Departments (nameDep, city) VALUES (p_nameDep, p_city);
END//
DELIMITER ;

CALL pPopulateDB();
CALL pInsertDepartment('Economía', 'Almeria');
-- Insertar departamento duplicado:
-- CALL pInsertDepartment ('Economía', 'Almeria');
```

Empleados/pInsertEmployee.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Procedimiento para insertar un nuevo Empleado, si la startDate es
null, poner la fecha actual del sistema
--

DELIMITER //
CREATE OR REPLACE PROCEDURE pInsertEmployee (
  p_employeeId INT,
  p_departmentId INT,
```

```
  p_bossId INT,
  p_nameEmp VARCHAR(64),
  p_startDate DATE,
  p_endDate DATE,
  p_salary DOUBLE,
  p_fee DOUBLE)
BEGIN
  IF (p_startDate IS NULL) THEN
    SET p_startDate = SYSDATE();
  END IF;
  INSERT INTO Employees (employeeId, departmentId, bossId, nameEmp, salary,
startDate, endDate, fee)
  VALUES (p_employeeId, p_departmentId, p_bossId, p_nameEmp, p_salary,
p_startDate, p_endDate, p_fee);
END //
DELIMITER ;

CALL pPopulateDB();
CALL pInsertEmployee(6, 1, NULL, 'Daniel', '2020-09-15', NULL, 2500.0, 0.2);
```

Realice un procedimiento para igualar las comisiones de todos los empleados al valor de la comisión media:

Empleados/pEquateFees.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Procedimiento para igualar las fees de todos los Employees a la
media de las fees.
--

DELIMITER //
CREATE OR REPLACE PROCEDURE
  pEquateFees()
BEGIN
  DECLARE avgFee DOUBLE;
  SET avgFee = (SELECT AVG(fee) FROM Employees);
  -- Modifica TODOS los Employees, no tiene WHERE
  UPDATE Employees SET fee = avgFee;
END //
DELIMITER ;

-- CALL pPopulateDB();
-- CALL pEquateFees();
```

Implemente un procedimiento que aplique un aumento a la comisión de un empleado en particular:

Empleados/pRaiseFee.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Procedimiento para aplicar un aumento dado a la comisión de
empleado concreto
--
```

```
DELIMITER //
CREATE OR REPLACE PROCEDURE
  pRaiseFee(id INT, amount DOUBLE)
BEGIN
  DECLARE e ROW TYPE OF Employees;
  DECLARE newFee DOUBLE;
  SELECT * INTO e -- el resultado del select lo almacena en la variable
    FROM Employees
    WHERE employeeId = id;
  SET newFee = e.fee + amount;
  UPDATE Employees
    SET fee = newFee
    WHERE employeeId = id;
END //
DELIMITER ;

-- CALL pPopulateDB();
-- CALL pRaiseFee(4,0.3);
```

15.2. Funciones

Implemente una función que devuelva el número de empleados de una localidad concreta:

Empleados/fNumEmployees.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Función que devuelve el numero de Employees de una localidad
--

DELIMITER //
CREATE OR REPLACE FUNCTION
  fNumEmployees(c VARCHAR(64)) RETURNS INT
BEGIN
  RETURN (
    SELECT COUNT(*)
    FROM Employees E JOIN Departments D
    ON (E.departmentId = D.departmentId)
    WHERE D.city = c
  );
END//
DELIMITER ;

SELECT *, fNumEmployees(d.city) numEmployees
FROM Departments d
;
```

Implemente una función que calcule la media de las comisiones de los empleados y use esa función dentro de un procedimiento almacenado para igualar las comisiones de todos los empleados al valor de la comisión media:

Empleados/fAvgFee.sql

```
--
-- Autor: David Ruiz
```

```
-- Fecha: Noviembre de 2022
-- Descripción: Ejemplo de uso de funciones dentro de procedimientos deben
almacenarse en variables
--

DELIMITER //
CREATE OR REPLACE FUNCTION
  fAvgFee() RETURNS DOUBLE
BEGIN
  RETURN (
    SELECT AVG(fee)
    FROM Employees
  );
END //
DELIMITER ;

-- El procedimiento puede usar la función
DELIMITER //
CREATE OR REPLACE PROCEDURE
  pEquateFees()
BEGIN
  DECLARE af DOUBLE;
  SET af = fAvgFee();
  UPDATE Employees SET fee = af;
END//
DELIMITER ;

-- CALL pPopulateDB();
-- CALL pEquateFees();
```

15.3. Cursores

Utilice un cursor para recorrer todos los empleados y calcular el valor acumulado de los salarios:

Empleados/fSumSalaries.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Ejemplo de función que recorre un cursor
--

DELIMITER //
CREATE OR REPLACE FUNCTION
  fSumSalaries() RETURNS DECIMAL
BEGIN
  DECLARE total DECIMAL;
  DECLARE employee ROW TYPE OF Employees;
  DECLARE done BOOLEAN DEFAULT FALSE;
  DECLARE curEmployees CURSOR FOR
    SELECT *
    FROM Employees;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done := TRUE;
  SET total = 0;
  OPEN curEmployees;
  readLoop: LOOP
    FETCH curEmployees INTO employee;
```

```
    IF done THEN
      LEAVE readLoop;
    END IF;
    SET total = total + employee.salary;
  END LOOP;
  CLOSE curEmployees;
  RETURN total;
END //
DELIMITER ;
```

15.4. Triggers (disparadores)

Implemente un disparador para evitar que un empleado sea su propio jefe:

Empleados/tSelfBoss.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Ejemplo de Trigger para comprobar que un empleado no puede ser su
propio boosId.
-- Dentro del Trigger se tiene acceso a una variable llamada 'new' que almacena la
tupla con
-- los datos a actualizar se usa SIGNAL para elevar error
--

DELIMITER //
CREATE OR REPLACE PROCEDURE  pCheckSelfBoss(
  p_employeeId INT,
  p_bossId INT)
BEGIN
  IF (p_employeeId = p_bossId) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
      'An employee cannot be his own boss';
  END IF;
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE TRIGGER tBISelfBoss
BEFORE INSERT ON Employees FOR EACH ROW
BEGIN
  CALL pCheckSelfBoss (NEW.employeeId, NEW.bossId);
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE TRIGGER tBUSelfBoss
BEFORE UPDATE ON Employees FOR EACH ROW
BEGIN
  CALL pCheckSelfBoss (NEW.employeeId, NEW.bossId);
END //
DELIMITER ;

-- CALL pPopulateDB();
-- CALL pInsertEmployee(p_employeeId, p_departmentId, p_bossId, p_nameEmp,
p_startDate, p_endDate, p_salary, p_fee)
```

Implemente un disparador que evite que modifique la comisión de un empleado en más de un 20%:

Empleados/tChangeFee.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Ejemplo de Trigger para comprobar que las variaciones sobre
-- la comisión de los Employees no puede cambiar en más 0.2 puntos
-- Dentro del Trigger se tiene acceso a 'old', que almacena la tupla
-- con los valores antes de cambiar.

-- OPCIÓN 1: No se permite realizar el cambio en la comisión
DELIMITER //
CREATE OR REPLACE TRIGGER tChangeFee1
BEFORE UPDATE ON Employees FOR EACH ROW
BEGIN
  IF((new.fee - old.fee) > 0.2 OR ((new.fee - old.fee) < -0.2)) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
      'A fee cannot increase or decrease more than 0.2';
  END IF;
END //
DELIMITER ;

-- OPCIÓN 2: Se permite realizar el cambio al valor máximo permitido
DELIMITER //
CREATE OR REPLACE TRIGGER tChangeFee2
BEFORE UPDATE ON Employees FOR EACH ROW
BEGIN
  IF((new.fee - old.fee) > 0.2) THEN
    SET new.fee = old.fee + 0.2;
  END IF;
  IF((new.fee - old.fee) < -0.2) THEN
    SET new.fee = old.fee - 0.2;
  END IF;
END //
DELIMITER ;
```

Implemente un disparador que evite que un departamento tenga más de cinco empleados:

Empleados/tMaxEmployeesDepartment.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Ejemplo de Trigger para comprobar que un Departamento
-- no tiene más de 5 Empleados.

DELIMITER //
CREATE OR REPLACE TRIGGER tMaxEmployeesDepartment
BEFORE INSERT ON Employees FOR EACH ROW
BEGIN
  DECLARE n INT;
  SET n = (
    SELECT COUNT(*)
```

```
    FROM Employees
    WHERE departmentId = new.departmentId
  );
  IF (n > 4) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
      'A department cannot have more than 5 employees';
    END IF;
  END //
DELIMITER ;

-- CALL pPopulateDB();
-- CALL pInsertEmployee(6, 1, NULL, 'quinto empleado departamento 1', null, NULL,
1500, 0);
-- CALL pInsertEmployee(7, 1, NULL, 'sexto empleado departamento 1', null, NULL,
1500, 0);
```

Implemente un disparador que en caso de insertar un empleado sin fecha de inicio, le ponga como fecha de inicio la fecha actual:

Empleados/tDefaultStartDate.sql

```
-- Ejemplo de Trigger para almacenar en la startDate del contrato
-- de un empleado a la fecha actual del sistema en que caso de que la
-- startDate proporcionada sea NULL
DELIMITER //
CREATE OR REPLACE TRIGGER tDefaultStartDate
BEFORE INSERT ON Employees FOR EACH ROW
BEGIN
  IF (new.startDate IS NULL) THEN
    SET new.startDate = SYSDATE();
  END IF;
END //
DELIMITER ;
```